МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №3

Выполнил:
студент группы ПО-12
Снарский П.П.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Вариант 12

Ход работы

Общее задание

• Прочитать задания, взятые из каждой группы, соответствующей одному из трех основных типов паттернов;

• Определить паттерн проектирования, который может использоваться при реализации задания. Пояснить свой выбор;

• Реализовать фрагмент программной системы, используя выбранный паттерн. Реализовать все необходимые дополнительные классы.

Задание 1. Порождающий паттерн

Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.

Код программы:

```
"""Abstract car factory"""

from abc import ABC, abstractmethod


class Sedan(ABC):
    """Abstract sedan class"""

    @abstractmethod
    def drive(self):
        """Drive the car"""

    @abstractmethod
    def honk(self):
        """Honk func"""


class MercedesSedan(Sedan):
    """Concrete sedan class"""

    def drive(self):
        print("Driving Mercedes Sedan")

    def honk(self):
        print("Honking Mercedes Sedan")


class AudiSedan(Sedan):
    """Concrete sedan class"""

    def drive(self):
        print("Driving Audi Sedan")

    def honk(self):
        print("Honking Mercedes Sedan")


class Coupe(ABC):
    """Abstract coupe class"""
```

```python
    @abstractmethod
    def drive(self):
        """Drive func"""

    @abstractmethod
    def honk(self):
        """Honk func"""


class MercedesCoupe(Coupe):
    """Concrete coupe class"""

    def drive(self):
        print("Driving Mercedes Coupe")

    def honk(self):
        print("Honking Mercedes Sedan")


class AudiCoupe(Coupe):
    """Concrete coupe class"""

    def drive(self):
        print("Driving Audi Coupe")

    def honk(self):
        print("Honking Mercedes Sedan")


class Factory(ABC):
    """Abstract factory class"""

    @abstractmethod
    def create_sedan(self) -> Sedan:
        """Create sedan func"""

    @abstractmethod
    def create_coupe(self) -> Coupe:
        """Create coupe func"""


class MercedesFactory(Factory):
    """Concrete factory class"""

    def create_sedan(self) -> Sedan:
        return MercedesSedan()

    def create_coupe(self) -> Coupe:
        return MercedesCoupe()


class AudiFactory(Factory):
    """Concrete factory class"""

    def create_sedan(self) -> Sedan:
        return AudiSedan()

    def create_coupe(self) -> Coupe:
        return AudiCoupe()


def test_drive(factory: Factory):
    """Client func"""
    sedan = factory.create_sedan()
    coupe = factory.create_coupe()

    sedan.drive()
```

```
    coupe.drive()


if __name__ == "__main__":

    print("Client test driving Mercedes:")
    test_drive(MercedesFactory())

    print("\n")

    print("Client test driving Audi:")
    test_drive(AudiFactory())
```

Результат работы:

```
Client test driving Mercedes:
Driving Mercedes Sedan
Driving Mercedes Coupe



Client test driving Audi:
Driving Audi Sedan
Driving Audi Coupe
```

Задание 2. Структурный паттерн

Проект «Универсальная электронная карта». В проекте должна быть реализована универсальная электронная карта, в которой есть функции паспорта, страхового полиса, банковской карты и т. д..

Код программы:

```
"""Facade universal card"""


class Passport:
    """Passport class"""
    def __init__(self, name, number):
        self.name = name
        self.number = number

    def get_info(self):
        """All passport info"""
        return f"Passport: {self.name}, №{self.number}"

    def get_number(self):
        """Show number"""
        return f"Passport №{self.number}"


class Insurance:
    """Insurance class"""
    def __init__(self, policy_number):
        self.policy_number = policy_number
        self.valid_time = 365
```

```python
    def get_number(self):
        """Insurance number"""
        return f"Insurance policy: №{self.policy_number}"

    def get_time(self):
        """Insurance time"""
        return f"Days till next insurance: {self.valid_time}"


class BankCard:
    """Bank card class"""
    def __init__(self, balance):
        self.balance = balance

    def pay(self, amount):
        """Pay money"""
        if amount <= self.balance:
            self.balance -= amount
            return f"Payment successful. Remaining balance: {self.balance}"
        return "Not enough funds"

    def get_balance(self):
        """Bank balance"""
        return f"Balance: {self.balance}"


class UniversalCard:
    """Facade universal card class"""
    def __init__(self, name, passport_number, policy_number, balance):
        self.passport = Passport(name, passport_number)
        self.insurance = Insurance(policy_number)
        self.bank = BankCard(balance)

    def show_all_info(self):
        """Show all info"""
        return "\n".join(
            [
                self.passport.get_info(),
                self.insurance.get_number(),
                self.bank.get_balance(),
            ]
        )

    def pay(self, amount):
        """Pay money"""
        return self.bank.pay(amount)


if __name__ == "__main__":
    card = UniversalCard("Ivan Ivanov", "1234 567890", "POL-001", 1000)

    print(card.show_all_info())
    print(card.pay(200))
```

Результат работы:

```
Passport: Ivan Ivanov, №1234 567890
Insurance policy: №POL-001
Balance: 1000
Payment successful. Remaining balance: 800
```

Задание 3. Поведенческий паттерн

Проект «Принтеры». В проекте должны быть реализованы разные модели принтеров, которые выполняют разные виды печати.

Код программы:

```
"""Strategy task"""
from abc import ABC, abstractmethod


class PrintStrategy(ABC):
    """Strategy abstract base class"""
    @abstractmethod
    def print(self, text):
        """Print text"""

    @abstractmethod
    def done(self):
        """Print done"""


class ColorPrint(PrintStrategy):
    """Strategy concrete class"""
    def print(self, text):
        return f"Color printing: {text}"

    def done(self):
        return "Color printing done"


class BWPrint(PrintStrategy):
    """Strategy concrete class"""
    def print(self, text):
        return f"Black & White printing: {text}"

    def done(self):
        return "BW printing done"


class FastPrint(PrintStrategy):
    """Strategy concrete class"""
    def print(self, text):
        return f"Fast printing: {text}"

    def done(self):
        return "Fast printing done"


class Printer:
    """Client class"""
    def __init__(self, strategy: PrintStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: PrintStrategy):
        """Set strategy"""
        self.strategy = strategy

    def print(self, text):
        """Print text"""
        return self.strategy.print(text)


if __name__ == "__main__":
    printer = Printer(ColorPrint())
```

```
print(printer.print("Hello World"))

printer.set_strategy(BWPrint())
print(printer.print("Hello World"))

printer.set_strategy(FastPrint())
print(printer.print("Hello World"))
```

Результат выполнения:

```
Color printing: Hello World
Black & White printing: Hello World
Fast printing: Hello World
```

Вывод: в ходе выполнения лабораторной работы приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python